

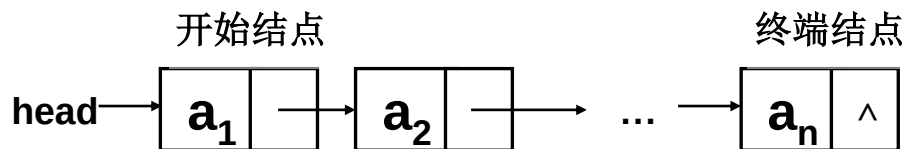
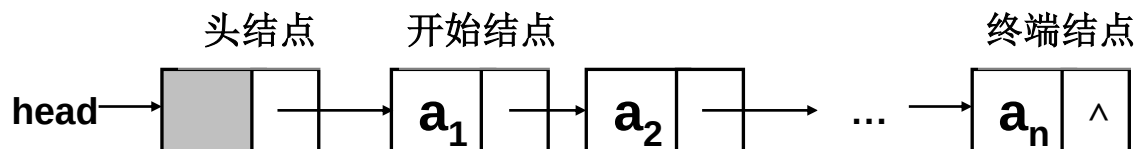
第二章 线性表

回顾



●**结点**：数据域 + 指针域（链域）

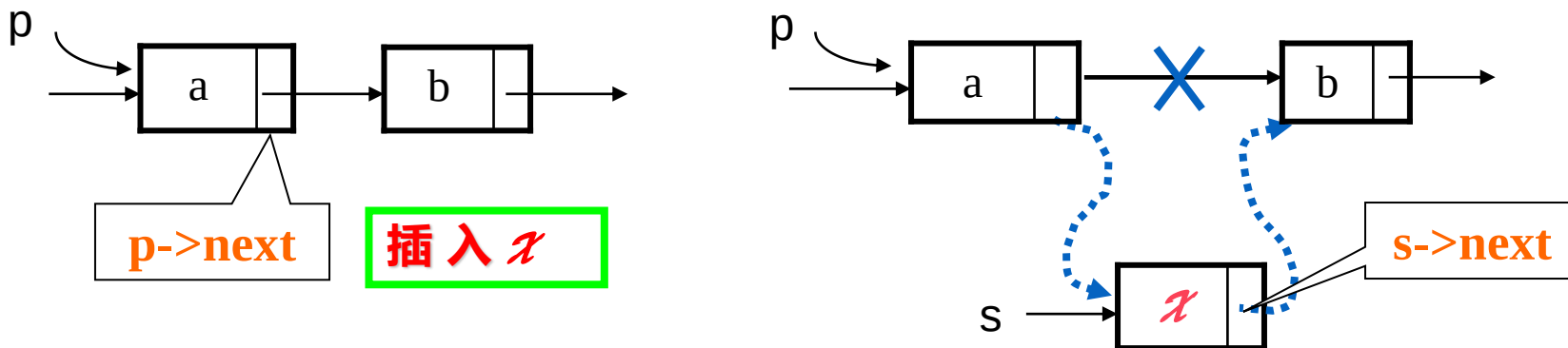
```
typedef struct Lnode {  
    ElemType    data;    //数据域  
    struct Lnode *next;  //指针域  
}Lnode, *LinkList;      // *LinkList为Lnode类型的指针
```



回顾

(2)插入元素

在链表中插入一个元素 x 的示意图如下：



链表插入的核心语句：

Step 1: $s \rightarrow next = p \rightarrow next;$
Step 2: $p \rightarrow next = s;$

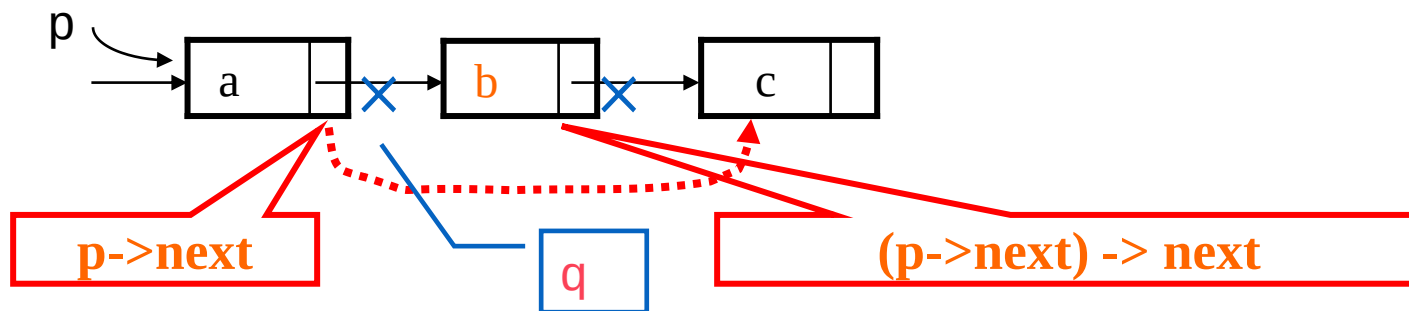
x 结点的生成方式：

```
s = (node*) malloc (m);  
s->data =  $x$ ;  
s->next = ?
```

2.3.1 线性表的链式存储—链表

(3)删除元素

设 p 指向单链表中某结点，删除 $*p$ 。操作示意图如图所示。要实现对结点 $*p$ 的删除，首先要找到 $*p$ 的前驱结点 $*q$ ，然后完成指针的操作即可。



删除动作的核心语句（要借助辅助指针变量 q ）：

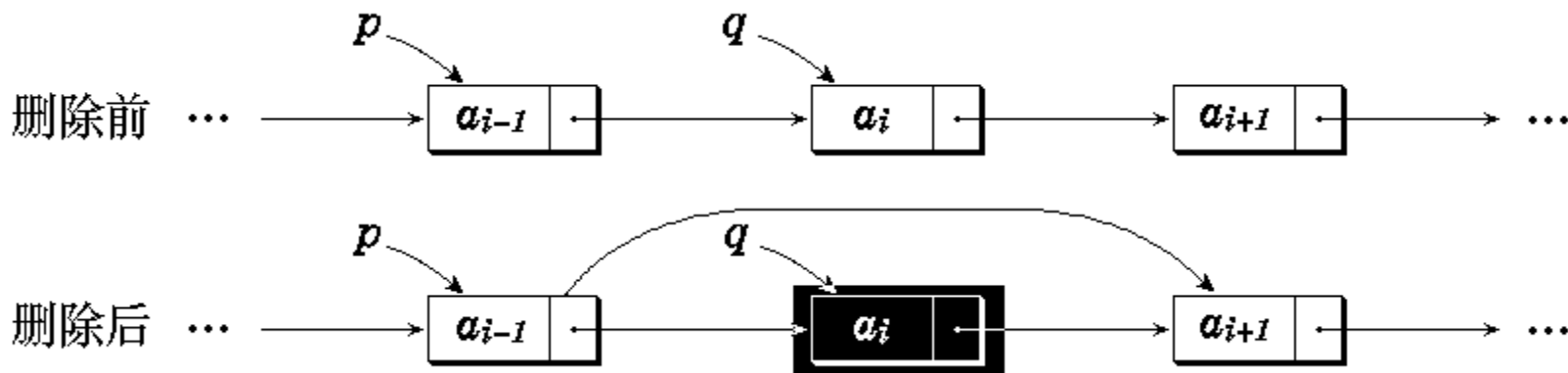
```
q = p->next;      //首先保存b的指针，靠它才能找到c；  
p->next=q->next;  //将a、c两结点相连，淘汰b结点；  
free(q);          //彻底释放b结点空间
```

思考：省略`free(q)`语句行不行？

2.3.1 线性表的链式存储—链表

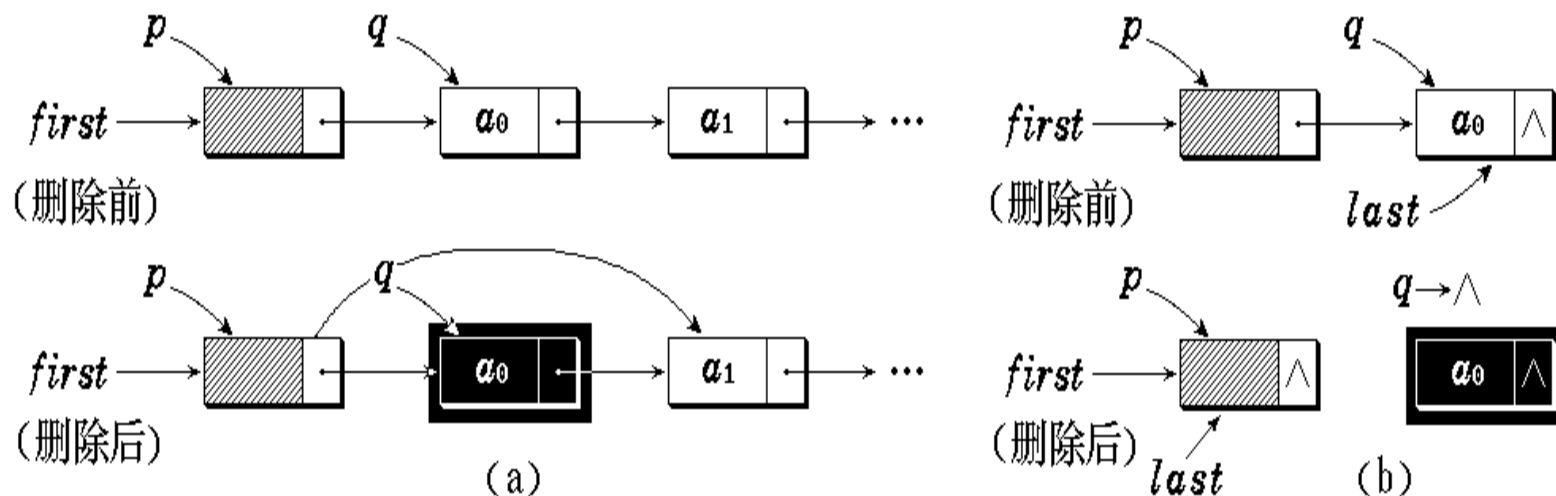
(3)删除元素

```
Status ListDelete_L(LinkList &L, int i, ElemType &e) {  
    LinkList p, q;  
    p=L;    int j=0;  
    while ( p->next && j<i-1 ) { p=p->next; ++j;}  
    if (!(p->next) || j> i-1) return ERROR;  
    q=p->next; p->next = q->next;  
    e=q->data; free(q);  
    return OK;  
}
```



2.3.1 线性表的链式存储—链表

从带表头结点的单链表中删除第一个结点



```
q = p→next;  
p→next = q→next;  
delete q;  
if ( p→next == NULL ) last = p;
```

2.3.1 线性表的链式存储—链表

插入和删除

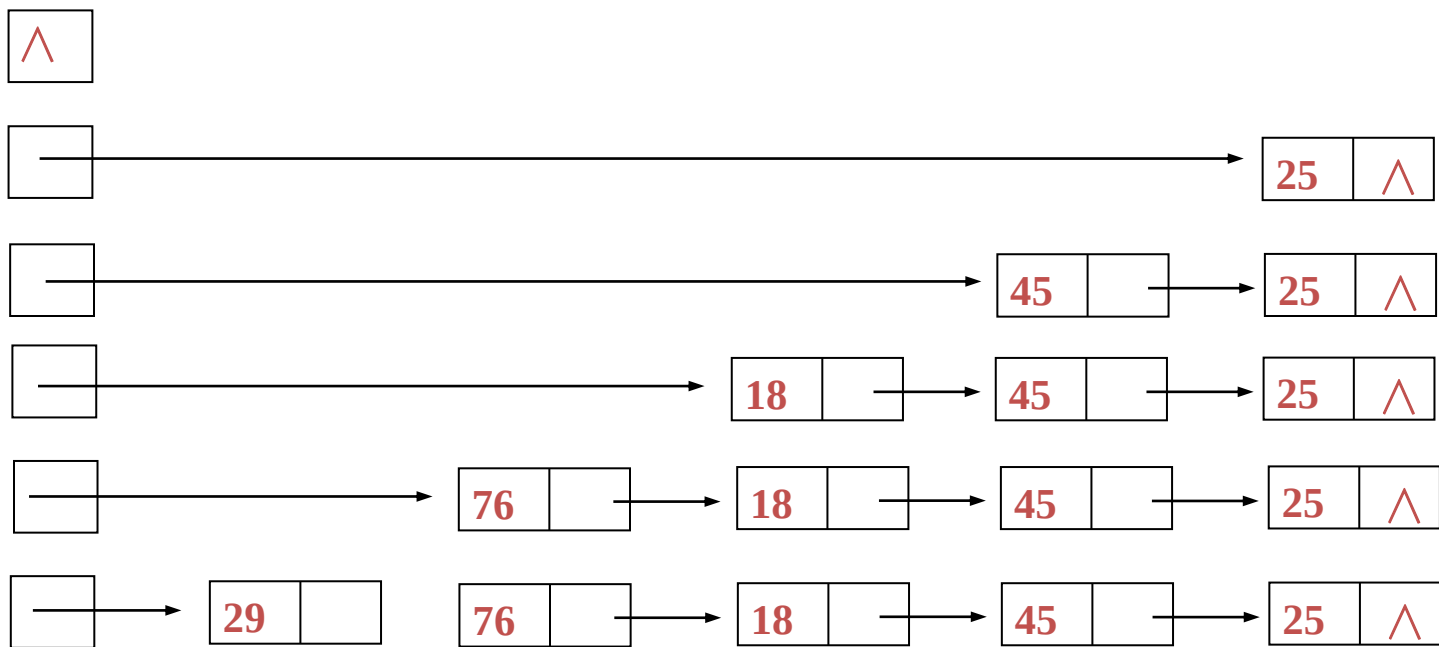
因线性链表不需要移动元素，只要修改指针，一般情况下时间复杂度为 $O(1)$ 。

如果要在单链表中进行前插或删除操作，因为要从头查找前驱结点，所耗时间复杂度将是 $O(n)$ 。

2.3.1 线性表的链式存储—链表

(4)建立单链表

□ 头插法建表：读取字符数组a中的字符,生成新结点,将读取的数据存放到新结点的数据域中,然后将新结点插入到当前链表的表头上,直到结束为止:



线性表：（25,45,18,76,29）之链表的建立过程

2.3.1 线性表的链式存储—链表

(4)建立单链表

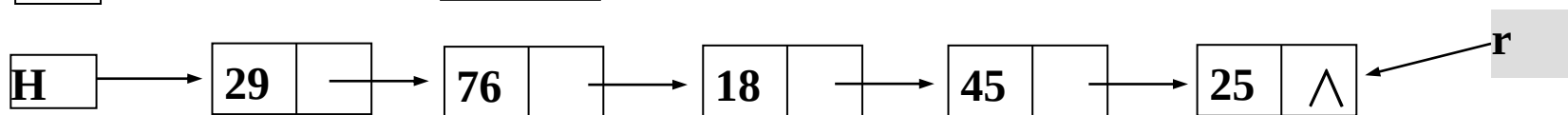
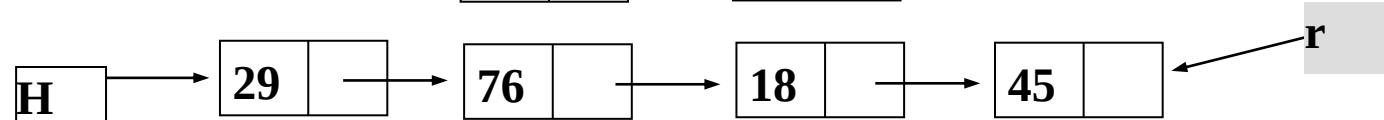
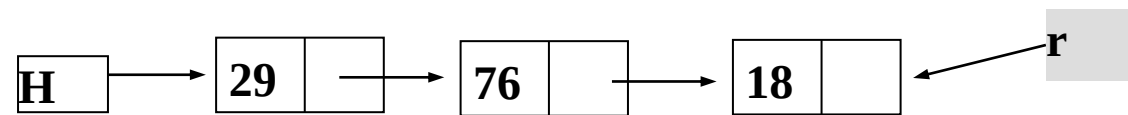
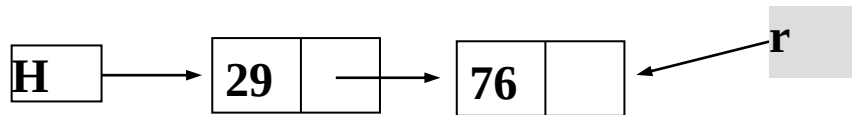
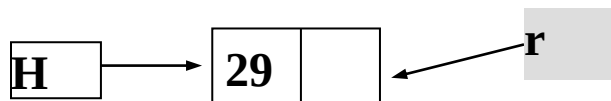
```
void CreateListL(LinkList &L,ElemType a[],int n) {  
    LNode *s;int i;  
    L=(LNode *)malloc(sizeof(LNode)); /*创建头结点*/  
    L->next=NULL;  
    for (i=0;i<n;i++){  
        s=(LNode *)malloc(sizeof(LNode));  
        /*创建新结点*/  
        s->data=a[i]; s->next=L->next;  
        /*将*s插入在原开始结点之前,头结点之后*/  
        L->next=s;  
    }  
}
```

2.3.1 线性表的链式存储—链表

□ 尾插法建表

将新结点插到当前链表的表尾上,为此必须增加一个尾指针 r ,使其始终指向当前链表的尾结点:

$H=NULL$ $r=NULL$ /*初始状态*/



线性表: (25,45,18,76,29) 之链表的建立过程

2.3.1 线性表的链式存储—链表

算法思路：初始状态，头指针 $H=NULL$ ，尾指针 $r=NULL$ ；按线性表中元素的顺序依次读入数据元素，不是结束标志时，申请结点，将新结点插入到 r 所指结点的后面，然后 r 指向新结点。

```
void CreateListR(LinkList &L,ElemType a[],int n){
    LNode *s,*r;int i;
    L=(LNode *)malloc(sizeof(LNode)); /*创建头结点*/
    r=L; /*r始终指向终端结点,开始时指向头结点*/
    for (i=0;i<n;i++){
        s=(LNode *)malloc(sizeof(LNode)); /*创建新结点*/
        s->data=a[i];r->next=s; /*将*s插入*r之后*/
        r=s;
    }
    r->next=NULL; /*终端结点next域置为NULL*/
}
```

2.3.1 线性表的链式存储—链表

(5)显示单链表

```
void DispList(LinkList L) { /*字母链表的输出*/
    p = L;
    while (p) { //当指针不空时循环，仅限于无头结点的情况
        printf("%c",p->data);
        p=p->next; //让指针不断“顺藤摸瓜”
    }
}
```

sum=0;

sum++;

讨论：要统计链表中数据元素的个数，该如何改写？

2.3.1 线性表的链式存储—链表

(6)单链表的修改(或读取)

思路：要修改第*i*个数据元素，必须从头指针起一直找到该结点的指针*p*，然后才能执行 `p->data=new_value` 。

修改第*i*个数据元素的操作函数可写为：

```
Status GetElem_L(LinkList L, int i, ElemType &e) {  
    p=L->next; j=1;    //带头结点的链表  
    while(p&& j<i) {p=p->next; ++j;}  
    if( !p ) return ERROR;  
    p->data =e; //若是读取则为： e=p->data;  
    return OK;  
} // GetElem_L
```

缺点：想寻找单链表中第*i*个元素，只能从头指针开始逐一查询（顺藤摸瓜），无法随机存取。

2.3.1 线性表的链式存储—链表

(7)初始化带头结点的线性表

该运算建立一个空的单链表，即创建一个头结点。

```
void InitList(LinkList &L)
{
    L=(LNode *)malloc(sizeof(LNode)); /*创建头结点*/
    L->next=NULL;
}
```

2.3.1 线性表的链式存储—链表

(8)销毁带头结点的线性表

释放单链表L占用的内存空间。即逐一释放全部结点的空间。

```
void DestroyList(LinkList &L) {  
    LNode *p=L,*q=p->next;  
    while (q!=NULL)  
    {    free(p);  
        p=q;q=p->next;  
    }  
    free(p);  
}
```

2.3.1 线性表的链式存储—链表

(9)判断带头结点的线性表是否为空表

```
int ListEmpty(LinkList L) {  
    return(L->next==NULL);  
}
```

2.3.1 线性表的链式存储—链表

(10)求带头结点的线性表长度

```
int ListLength(LinkList L) {  
    LNode *p=L;int i=0;  
    while (p->next!=NULL) {  
        i++;  
        p=p->next;  
    }  
    return(i);  
}
```

2.3.1 线性表的链式存储—链表

(11)查找带头结点线性表中的元素

```
int LocateElem(LinkList L,ElemType e) {  
    LNode *p=L->next;int n=1;  
    while (p!=NULL && p->data!=e) {  
        p=p->next; n++;  
    }  
    if (p==NULL) return(0);  
    else return(n);  
}
```

例

合并有序链表

算法要求：

已知：线性表 A 和 B，分别由单链表 L_a 和 L_b 存储，其中数据元素按值非递减有序排列（即已经有序）；

要求：将 A 和 B 归并为一个新的线性表 C，C 的数据元素仍按值非递减排列。设线性表 C 由单链表 L_c 存储。

假设：A = (3, 5, 8, 11)，B = (2, 6, 8, 9, 11)

预测：合并后的 C = (2, 3, 5, 6, 8, 8, 9, 11, 11)

例

算法主要包括搜索、比较、插入三个操作：

搜索：需要设立三个指针来指向La、Lb和Lc链表；

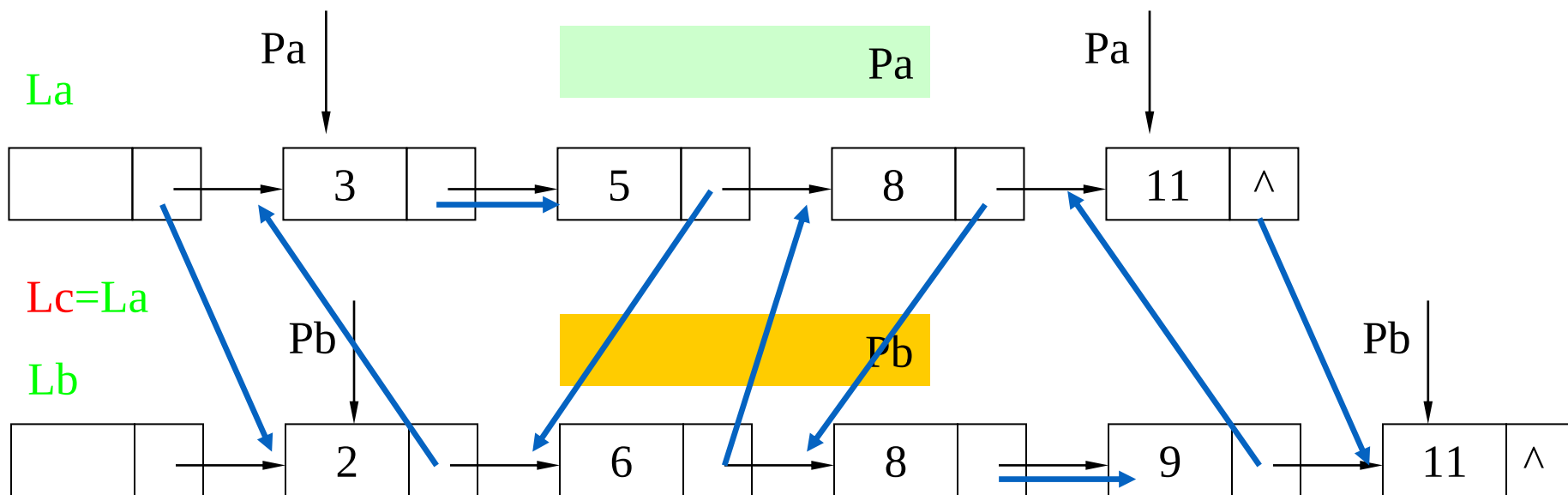
比较：比较La和Lb表中结点数据的大小；

插入：将La和Lb表中数据较小的结点插入新链表Lc。

请注意链表的特点，仅改变指针便可实现数据的移动，即
“数据不动，指针动”

例

P_a 、 P_b 用于搜索 L_a 和 L_b ， P_c 指向新链表当前结点。



例

```
void MergeList_L(LinkList &La, LinkList &Lb, LinkList &Lc) {
    LinkList pa, pb, pc;
    pa = La->next; pb = Lb->next;
    Lc = pc = La;
    while (pa && pb) {
        if (pa->data <= pb->data) {
            pc->next = pa; pc = pa; pa = pa->next;
        }
        else {pc->next = pb; pc = pb; pb = pb->next;}
    }
    pc->next = pa ? pa : pb;
    free(Lb);
}
```

不用Lc，直接把La表插到Lb表中；或者把Lb表插到La表中，怎么修改？

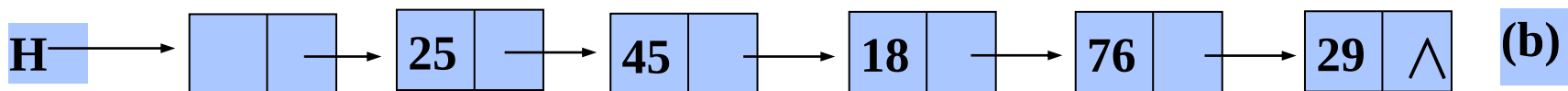
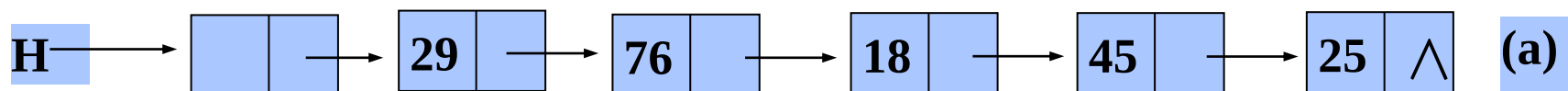
例

合并 $A=A \cup B$

```
void UnionList_L(LinkList &La, LinkList Lb) {
    LinkList p, q, first;  int x;
    first = La->next;  p=Lb->next;
    while (p) {
        x=p->data;  p=p->next;  q=first;
        while (q && q->data !=x)  q=q->next;
        if (!q) {
            q=(LinkList)malloc(sizeof(LNode));
            q->data = x;  q->next = La->next;
            La->next = q;
        }
    }
}
```

例

例 已知单链表H，写一算法将其倒置。即实现如图2.22的操作。(a)为倒置前，(b)为倒置后。



算法思路：

依次取原链表中的每个结点，将其作为第一个结点插入到新链表中去，指针p用来指向当前结点，p为空时结束。

例

```
void reverse (Linklist H) {  
    LNode *p;  
    p=H->next;  
    H->next=NULL;  
    while (p) {  
        q =p;  
        p=p->next;  
        q->next=H->next; H->next=q;  
    }  
}
```

例

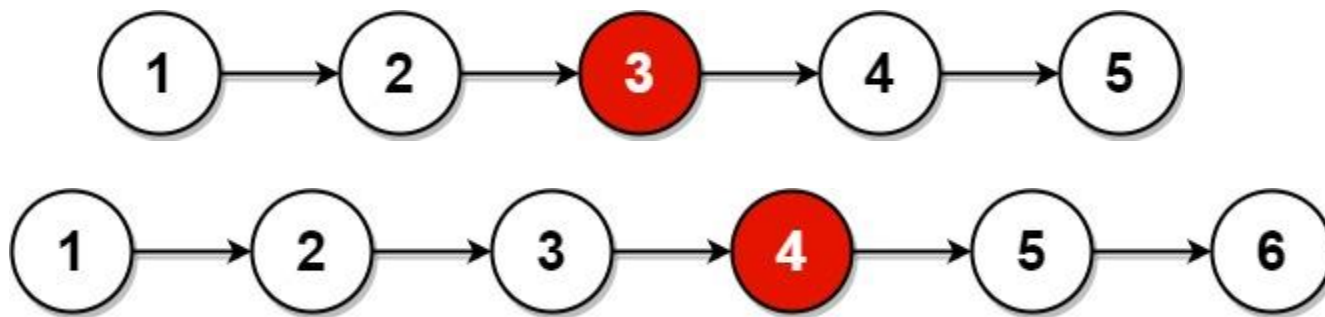
例：已知L为带头结点的单链表,请依照递归思想实现下列运算。

- 1、求链表中的最大整数
- 2、求表中的节点数?
- 3、求链表所有元素的平均值?

```
int getmax(Linklist L) {  
    if (L == NULL) {  
        return MIN;//宏定义MIN为一个很小的数例如-999999  
    }  
    int tmp = getmax(L->next);//获得下一个元素  
    return tmp > L->data ? tmp : L->data;//比较两个元素大小  
}
```

例

例 给你单链表的头结点 head，请你找出并返回链表的中间结点。如果有两个中间结点，则返回第二个中间结点。



思路：快慢指针的典型运用

快慢指针：慢指针每次移动一个结点，快指针每次移动两个结点，当快指针到达NULL时，慢指针刚好就在中间结点的位置，即 $2/n$ 的位置。

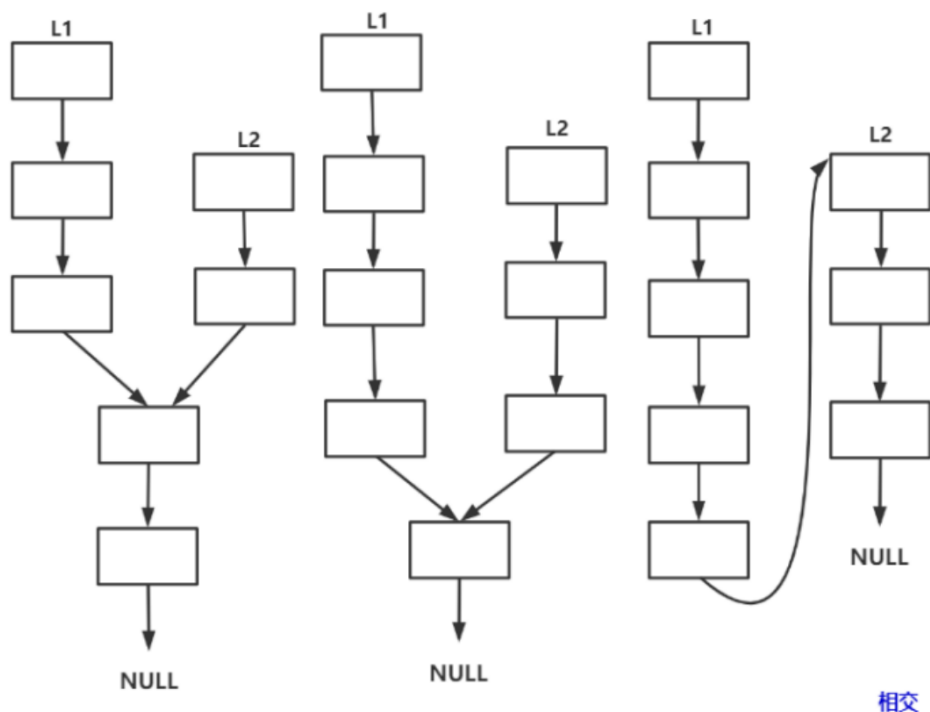
例

```
Lnode* middleNode(Linklist head) {  
    Lnode* slow = head;  
    Lnode* fast = head;  
    while(fast && fast->next)  
        //偶数个结点的判断条件是fast == NULL;  
        //奇数个结点的判断条件是fast->next == NULL;  
    {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
    return slow;  
}
```

例

例：输入两个链表，找出它们的第一个公共结点。

检测两个链表是否相交。分别找到两个链表中的最后一个节点，然后检测这两个节点的地址是否相同即可如果地址相同则相交，否则不相交



第2章 线性表

- 2.1 线性表的类型定义
- 2.2 线性表的顺序表示和实现
- 2.3 线性表的链式表示和实现
 - 2.3.1 线性链表——静态链表
- 2.4 一元多项式的表示及相加

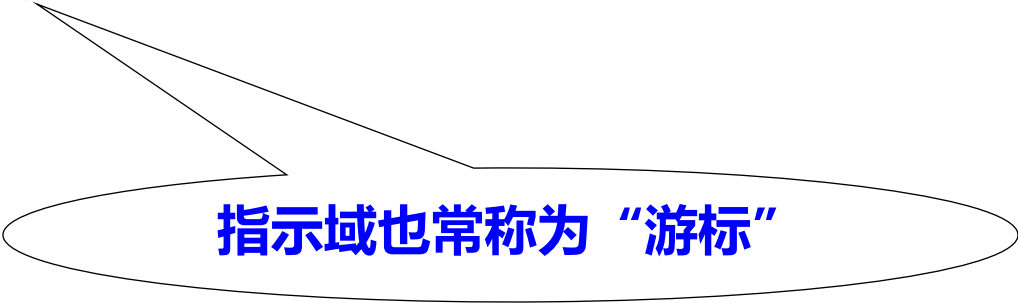
2.3.5 静态链表

问题：若某种高级语言没有指针类型，能否实现链式存储和运算？如何实现？

答：能！虽然链表通常用动态级联方式存储，但也可以用一片连续空间（一维数组）实现链式存储，这种方式称为**静态链表**。

2.3.5 静态链表

具体实现方法： 定义一个结构型数组（每个元素都含有**数据域**和**指示域**），就可以完全描述链表，**指示域**就相当于动态链表中的指针。

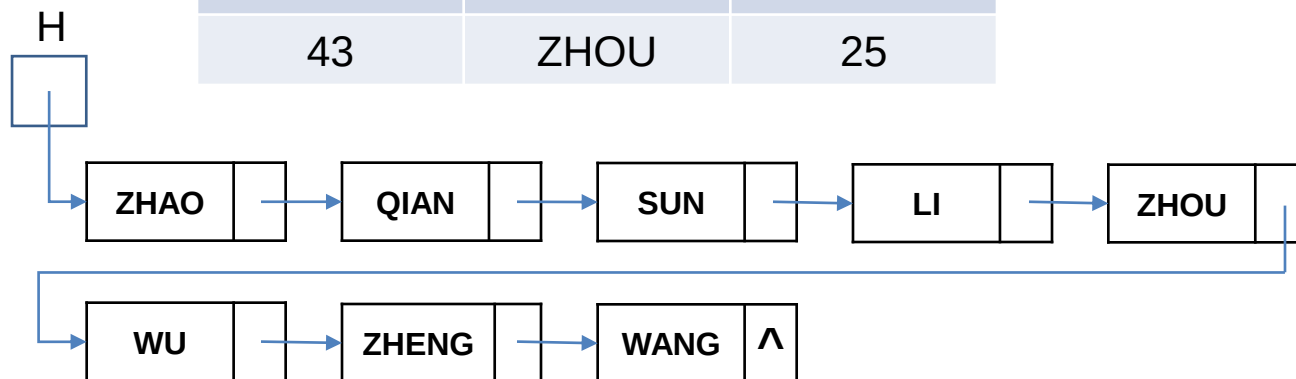


指示域也常称为“游标”

2.3.5 静态链表

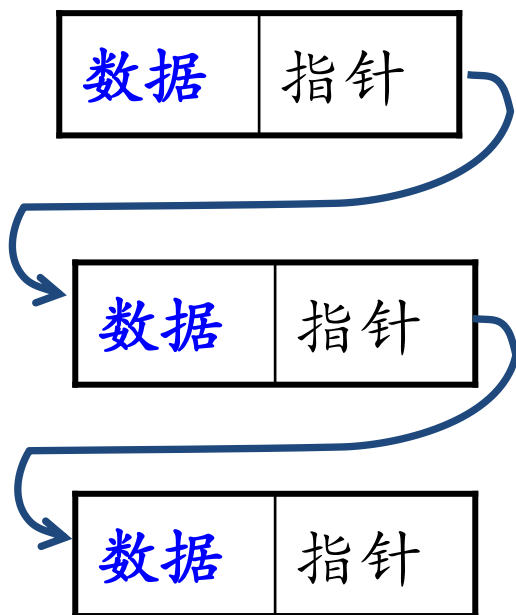
例： (ZHAO, QIAN, SUN, LI, ZHOU, WU, ZHENG, WANG)

存储地址	数据域	指针域
1	LI	43
7	QIAN	13
13	SUN	1
19	WANG	NULL
25	WU	37
31	ZHAO	7
37	ZHENG	19
43	ZHOU	25



2.3.5 静态链表

动态链表样式：



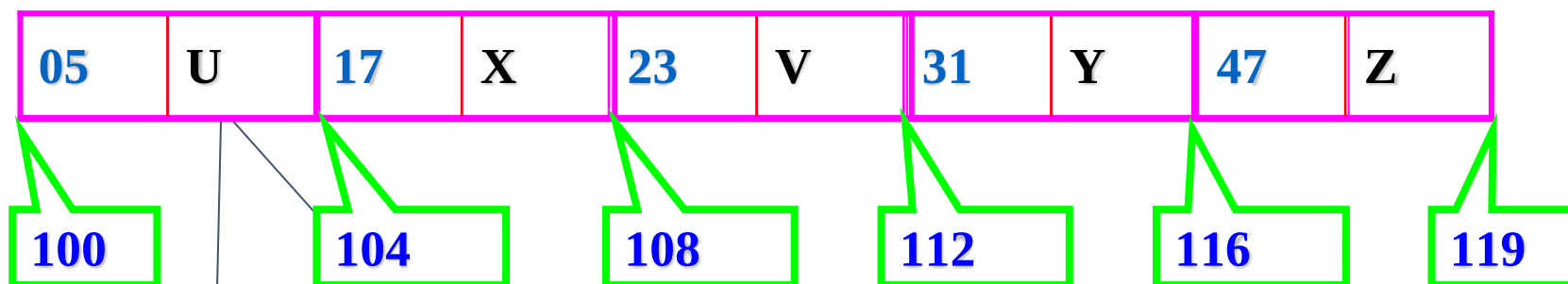
静态链表样式：

数据	指示
数据	指示
数据	指示
数据	指示
数据	指示

数组中每个元素都至少有两个分量，属于结构型数组。

常用于无指针类型的高级语言中。

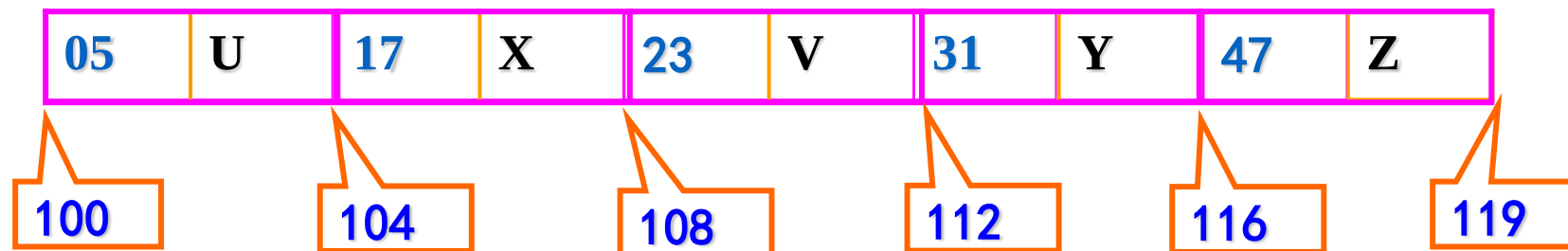
2.3.5 静态链表



若此分量定义为指针型，属于动态链表；
若此分量定义为整型（通常存放下标），则属于静态链表。

2.3.5 静态链表

现有一个具有五个元素的线性表 $L=\{23, 17, 47, 05, 31\}$ ，若它以下列方式存储在下列100~119号地址空间中，每个结点由数据（占2个字节）和指针（占2个字节）组成。



其中指针X, Y, Z的值分别为多少？该线性表的首结点起始地址为多少？末结点的起始地址为多少？

答：X= 116 Y= NULL Z= 100,
首址= 108 末址= 112。

2.3.5 静态链表

静态单链表的类型定义如下：

```
#define MAXSIZE 1000
```

```
//预分配最大的元素个数（连续空间）
```

```
typedef struct {
```

```
    ElemType data; //数据域
```

```
    int cur; //指示域
```

```
}component, SLinkList[MAXSIZE];
```

```
//这是一维结构型数组
```

2.3.5 静态链表

前例：一线性表 $S = (\text{ZHAO}, \text{QIAN}, \text{SUN}, \text{LI}, \text{ZHOU}, \text{WU})$ ，用静态链表如何表示？

i	data	cur
0	头结点	1
1	ZHAO	3
2	LI	5
3	QIAN	6
4	WU	0
5	ZHOU	4
6	SUN	2
...		
1000		

说明1：假设S为SLinkList型变量，则

S[MAXSIZE] 为一个静态链表；

S[0].cur则表示第1个结点在数组中的位置。

说明2：如果数组的第i个分量表示链表的第k个结点，则：

S[i].data表示第k个结点的数据；

S[i].cur 表示第k+1个结点(即k的直接后继)的位置。

2.3.5 静态链表

□ 初始化静态链表

```
int InitSpace_ SL(SLinkList &space) {  
    for(i = 0; i<MAXSIZE-1; ++i)  
        space[i].cur = i+1;  
    space[MAXSIZE- 1]. cur = 0;  
}
```

□ 静态链表查找

```
int LocateElem_ SL(SLink:List S, ElemType e) {  
    i = S[0].cur;  
    while (i && S[i].data != e)  
        i = S[i].cur;  
    return i;  
}
```

2.3.5 静态链表

例如：在线性表 $S = (ZHAO, QIAN, SUN, LI, ZHOU, WU)$ 的 **QIAN, SUN** 之间插入新元素 **LIU**，可以这样实现：

i	data	cur
0	头结点	1
1	ZHAO	3
2	LI	5
3	QIAN	7
4	WU	0
5	ZHOU	4
6	SUN	2
7	LIU	6
1000	...	...

Step1: 将QIAN的游标值存入LIU的游标中：

$S[7].cur = S[3].cur;$

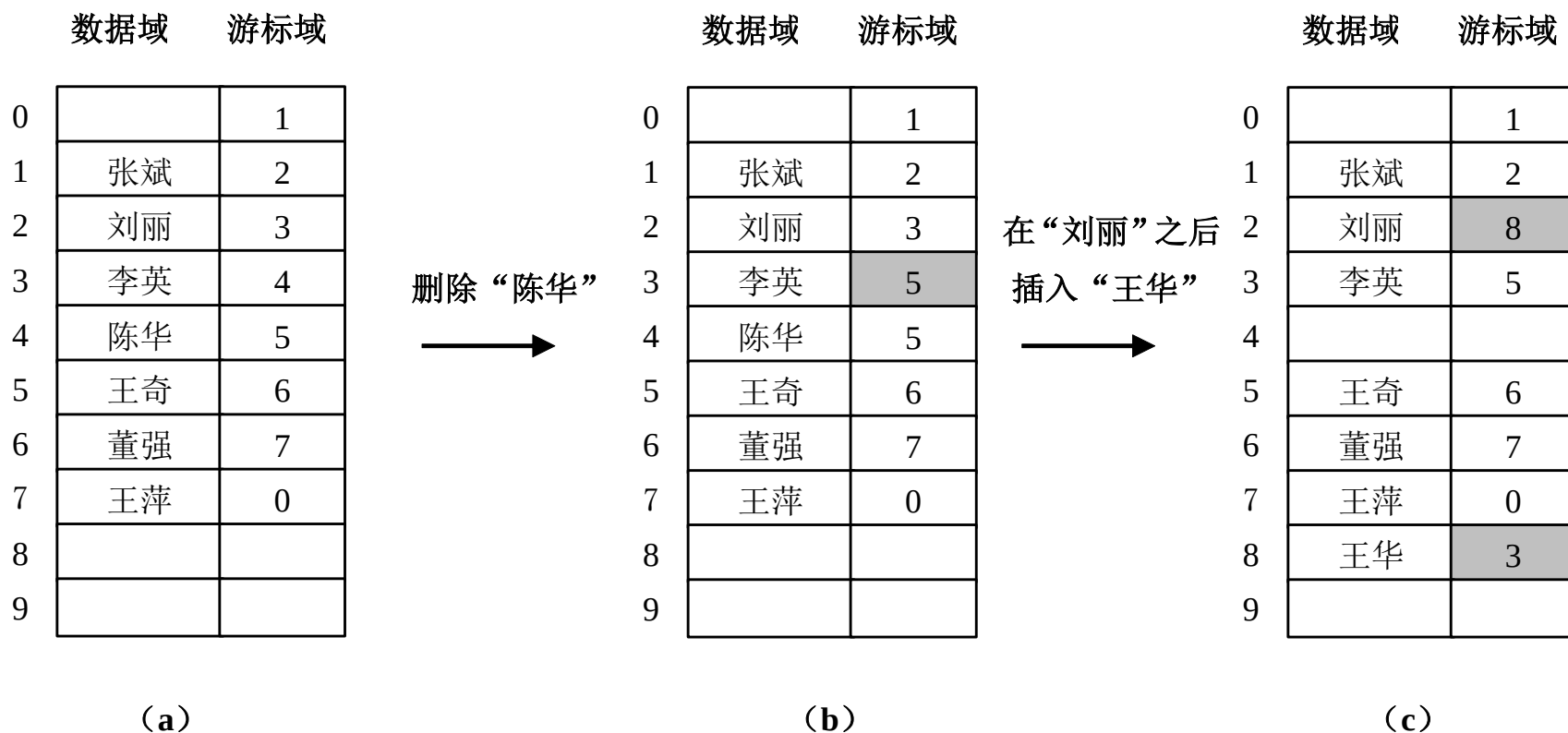
Step2: 将QIAN的游标换为新元素LIU的下标：

$S[3].cur = 7$

说明3：静态链表的插入与删除操作与普通链表一样，不需要移动元素，只需修改指示器就可以了。

2.3.5 静态链表

下图给出了一个静态链表的示例。图(a)是一个修改之前的静态链表，图(b)是删除数据元素“陈华”之后的静态链表，图(c)插入数据元素“王华”之后的静态链表，图中用阴影表示修改的游标。



2.3.5 静态链表

例：集合运算(A-B) U (B-A)

假设由终端输入集合元素，先建立表示集合A的静态链表S, 而后在输入集合B的元素的同时查找S表，若存在和B相同的元素，则从S表中删除之，否则将此元素插入S表。

A = (c, b, e, g, f, d)

B = (a, b, n, f)

space(0 : 11)		
0		8
1		2
2	c	3
3	b	4
4	e	5
5	g	6
6	f	7
7	d	0
8		9
9		10
10		11
11		0

(a)



space(0 : 11)		
0		6
1		2
2	c	4
3	n	8
4	e	5
5	g	7
6	f	9
7	d	3
8	a	0
9		10
10		11
11		0

(b)

2.3.5 静态链表

□ 初始化静态链表

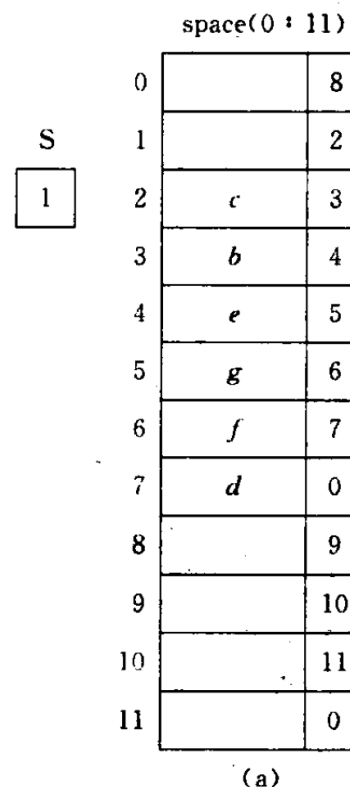
```
void InitSpace_ SL(SLinkList &space){  
    for(i = 0; i<MAXSIZE-1; ++i)    space[i].cur = i+1;  
    space[MAXSIZE- 1]. cur = 0;  
}
```

□ 分配存储区域

```
int Malloc_ SL(SLinkList &space){  
    i = space[0].cur;  
    if (space[0].cur)    space[0].cur = space[i].cur;  
    return i;  
}
```

□ 释放存储区域

```
int Free_ SL(SLinkList &space, int k){  
    space[k].cur = space[0].cur;    space[0].cur = k;  
}
```



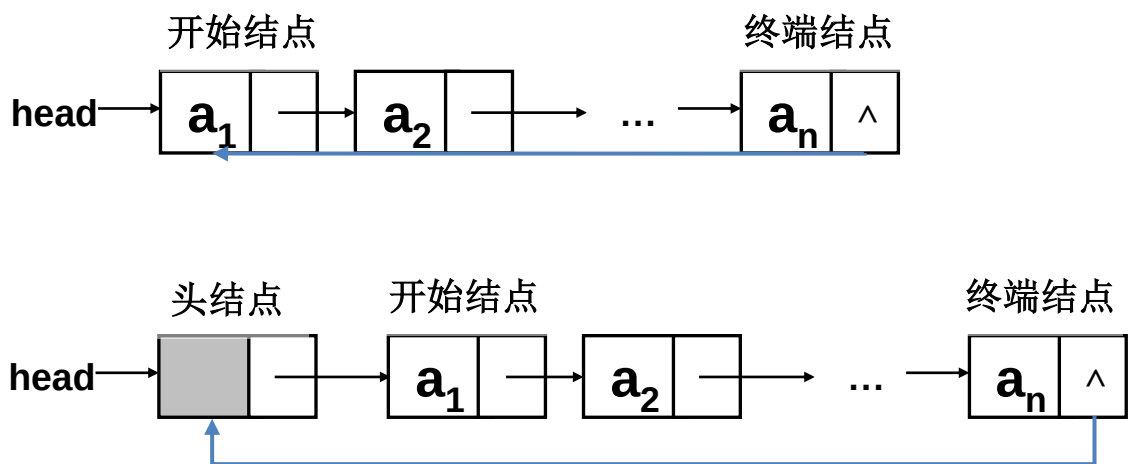
2.3.5 静态链表

```
void difference(SLinkList &space, int &S){
    InitSpace_SL(space); S = Malloc_SL(space); r = S;
    scanf(m, n);
    for (j = 1; j <= m; ++ j) {
        i = Malloc_SL(space); scanf(space[i].data); space[r].cur= i; r = i;
    }
    space[r].cur = 0;
    for(j = 1; j <= n; ++ j) {
        scanf(b); p = S; k = space[S].cur;
        while (k != space[r].cur && space[k].data != b){
            p = k; k = space[k].cur;
        }
        if (k == space[r].cur) {
            i = Malloc_SL(space); space[i].data = b; space[i].cur = space[r].cur;
            space[r].cur = i;
        }
        else{ space[p].cur = space[k].cur; Free_SL(space, k); if (r == k) r = p;}
    }
}
```

第2章 线性表

- 2.1 线性表的类型定义
- 2.2 线性表的顺序表示和实现
- 2.3 线性表的链式表示和实现
 - 2.3.1 线性链表
 - 2.3.2 循环链表
- 2.4 一元多项式的表示及相加

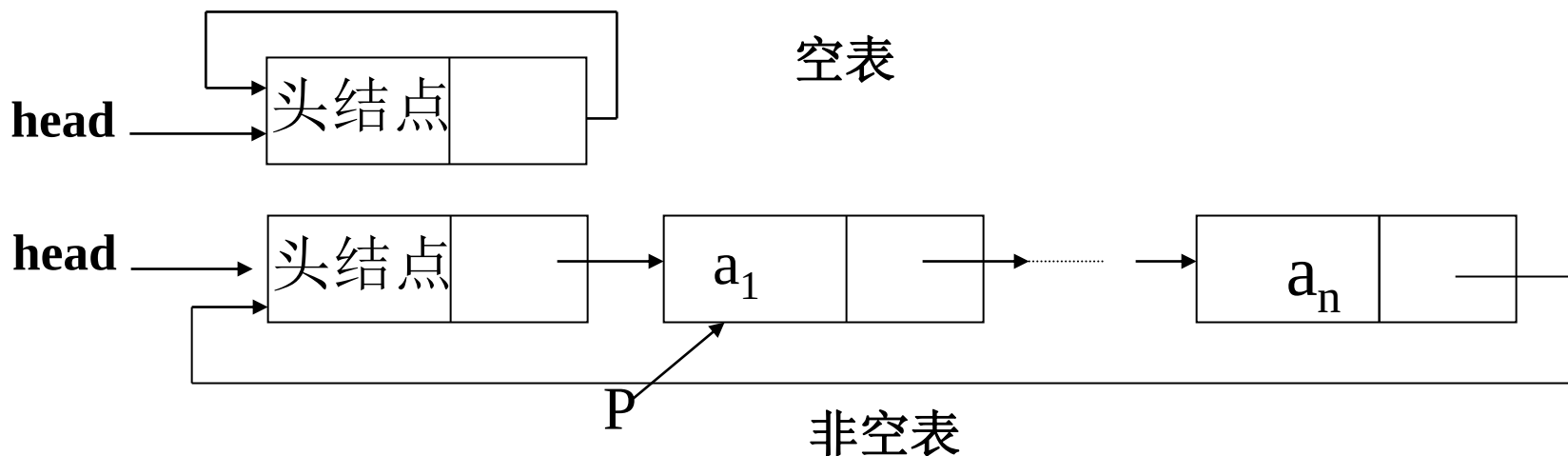
2.3.2 循环链表



特点

- (1) 尾结点指针域指向头结点
- (2) 结点结构与线性链表相同
- (3) 有关操作与线性链表基本相同，仅有判断链表尾方法不同
- (4) 知道任何一个指向某一结点的指针就可以访问链表中的所有结点

2.3.2 循环链表



(1) 判断链表空吗?

$p \rightarrow next == p$, 则链表为空或者只有一个元素

(2) 判断链表遍历完否?

$p \rightarrow next == head$, 则遍历完毕

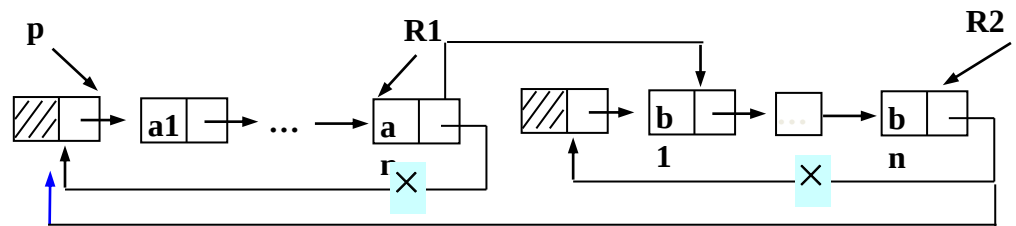
(3) 插入或删除结点

与一般的单向链表相同

2.3.2 循环链表

(3) 合并两个链表

单链表: $p = La;$
 $\text{while } (p \rightarrow \text{next}) \ p = p \rightarrow \text{next};$
 $p \rightarrow \text{next} = Lb \rightarrow \text{next};$
 $\text{free}(Lb);$



循环链表: $p = La;$
 $\text{while } (p \rightarrow \text{next} \neq La) \ p = p \rightarrow \text{next};$
 $p \rightarrow \text{next} = Lb \rightarrow \text{next};$
 $p = p \rightarrow \text{next};$
 $\text{while } (p \rightarrow \text{next} \neq Lb) \ p = p \rightarrow \text{next};$
 $p \rightarrow \text{next} = La;$
 $\text{free}(Lb);$

2.3.2 循环链表

(4) 建立循环链表

```
void CreateList_L(LinkList &L) {  
    LNode* p;  int x;  
    L= (LinkList)malloc(sizeof(LNode));  
    L->next=L;  
    while (scanf("%d", &x), x!=0 ) {  
        p=(LNode*)malloc(sizeof(LNode));  
        p->data=x; p->next = L->next;  
        L->next = p;  
    }  
}
```

2.3.2 循环链表

(5) 显示输出循环链表

```
void PrintList_LC(LinkList L)
{ LinkList p;
  p=L->next;  printf("L->");
  while (p!=L) {
    printf("%d->", p->data);
    p=p->next;  }
  printf("L\n");
}
```

2.3.2 循环链表

例 将一循环单链表就地逆置。

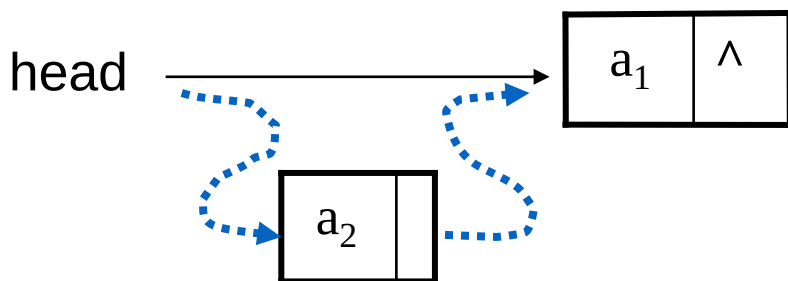
分析：要想让 a_n 指向 $a_{n-1}, \dots, a_2$ 指向 a_1 ，一般有两种算法：

① **替换法：**扫描 $a_1, \dots, a_n$ ，将每个 a_i 的指针域指向 a_{i-1} 。

思路：后继变前驱

② **插入法：**扫描 $a_1, \dots, a_n$ ，将每个 a_i 插入到链表首部即可。

思路：头部变尾部



**实际上是链栈
的概念**

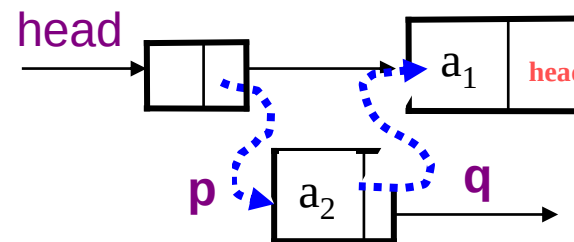
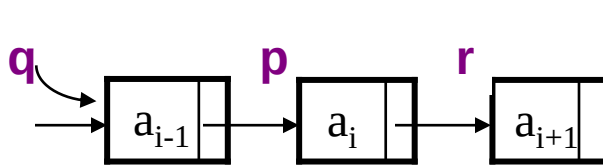
2.3.2 循环链表

替换法的核心语句:

```
q=head; //逆置后的第一个结点
p=head->next; //有头结点
while(p!=head) //循环单链表
{ r=p->next;
  p->next=q; //前驱变后继
  q=p;
  p=r; } //准备处理下一结点
head->next=q; //以 $a_n$ 为首
```

插入法的核心语句:

```
p=head->next; //有头结点
if(p!=head){r=p->next;
  p->next=head;p=r}; //处理 $a_1$ 
while(p!=head) //循环单链表
{ r=p->next //保存原后继
  p->next=head->next;
  head->next=p;
  p=r;} //准备处理下一结点
```



请上机验证并分析效率!

第2章 线性表

- 2.1 线性表的类型定义
- 2.2 线性表的顺序表示和实现
- 2.3 线性表的链式表示和实现
 - 2.3.1 线性链表
 - 2.3.2 循环链表
 - 2.3.3 双向链表
- 2.4 一元多项式的表示及相加

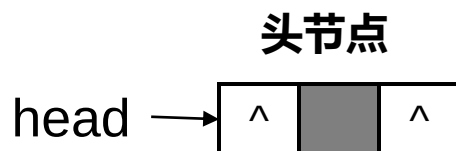
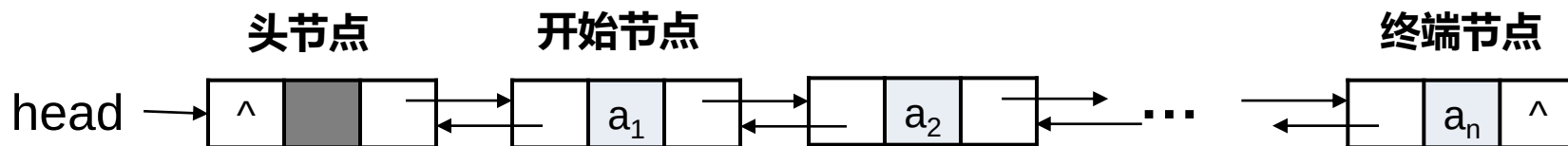
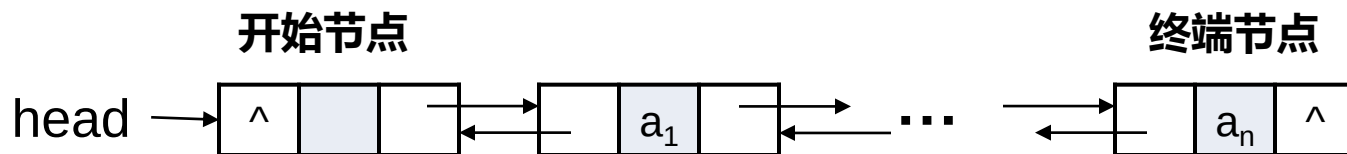
2.3.4 双链表

双向链表(Double linked list):在单链表的每个结点里再增加一个指向其直接前趋的指针域prior。这样就形成的链表中有两个方向不同的链，故称为双向链表。

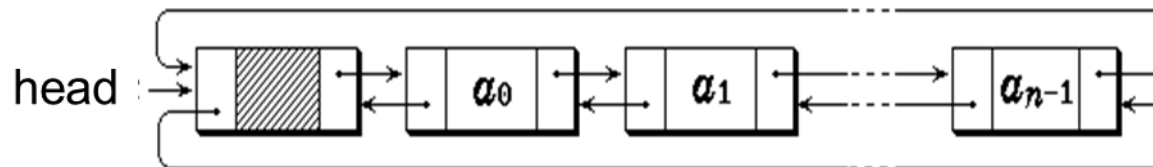


```
typedef struct DNode { /*定义双链表结点类型*/  
    ElemType data;  
    struct DNode *prior; /*指向前驱结点*/  
    struct DNode *next;  /*指向后继结点*/  
} DLinkList;
```

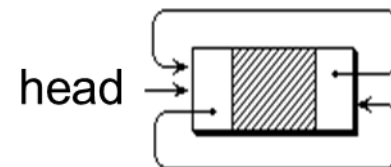
2.3.4 双链表



2.3.4 双链表



(b) 非空双向循环链表



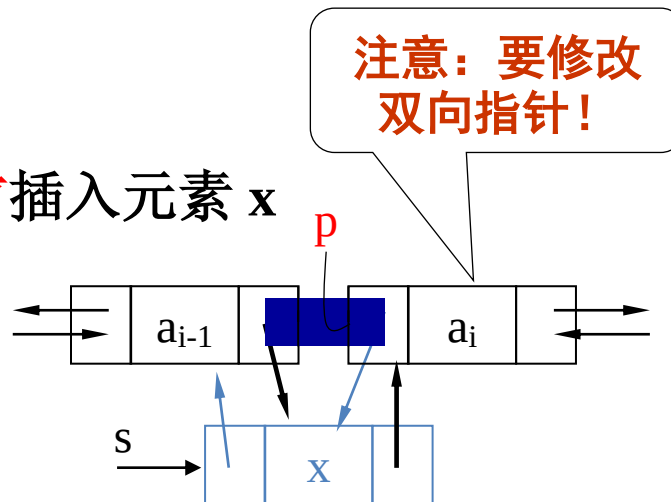
(c) 空表

- 对双向循环链表中任一结点的指针，有：
$$p == p \rightarrow \text{prior} \rightarrow \text{next} == p \rightarrow \text{next} \rightarrow \text{prior}$$
- 置空表：
$$p \rightarrow \text{prior} = p ; p \rightarrow \text{next} = p ;$$

2.3.4 双链表

(1) 双向链表插入

设 p 已指向第 i 元素，请在第 i 元素前插入元素 x



指针域的变化：

① a_{i-1} 的后继从 a_i (指针是 p)变为 x (指针是 s):

$s \rightarrow \text{next} = p$; $p \rightarrow \text{prior} \rightarrow \text{next} = s$;

② a_i 的前驱从 a_{i-1} (指针是 $p \rightarrow \text{prior}$)变为 x (指针是 s);

$s \rightarrow \text{prior} = p \rightarrow \text{prior}$; $p \rightarrow \text{prior} = s$;

2.3.4 双链表

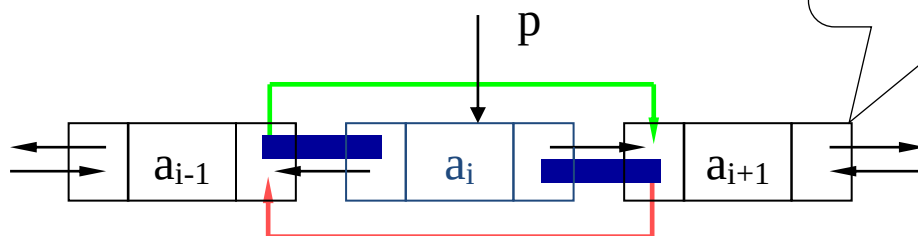
(1) 双向链表插入

```
void ListInsert(DuLNode* p, ElemType x)
{
    //申请新节点
    DuLNode* s = (DuLNode*)malloc(sizeof(DuLNode));
    //1.将新节点的next改为当前节点
    s->next = p;
    //2.将当前节点的上一节点的next改为新节点
    p->prior->next = s;
    //3.将新节点的prior改为当前节点的上一节点
    s->prior = p->prior;
    //4.将当前节点的prior改为新节点
    p->prior = s;
}
```

2.3.4 双链表

(2) 双向链表删除

设 p 指向第 i 个元素，删除第 i 个元素



指针域的变化：

后继方向： a_{i-1} 的后继由 a_i (指针 p)变为 a_{i+1} (指针 $p \rightarrow next$) ;

$p \rightarrow prior \rightarrow next = p \rightarrow next$;

前驱方向： a_{i+1} 的前驱由 a_i (指针 p)变为 a_{i-1} (指针 $p \rightarrow prior$) ;

$p \rightarrow next \rightarrow prior = p \rightarrow prior$;

2.3.4 双链表

(2) 双向链表删除

```
void ListDelete(DuLNode* p)
{
    //禁止删除头节点
    assert(p->next != p);
    //1.当前位置的上一节点的next变为当前节点的下一节点
    p->prior->next = p->next;
    //2.当前位置的下一节点的prior变为当前节点的上一节点
    p->next->prior = p->prior;
    //释放当前位置的节点
    free(p);
    p = NULL;
}
```

2.3.4 双链表

删除第一个数据域为x的节点

```
void ListDeleteByData(DuLNode* L, ElemType x){
    DuLNode* p,q,r;
    p = L;
    while(p->next && p->next->data != x){
        p = p->next;
    }
    if(!(p->next)){
        printf("The char '%c' doesn't exist.\n", x);
        return ;
    }
    q = p->next;
    r = q->next;
    p->next = r;
    if(r){
        r->prior = p;
    }
    free(q);
}
```

正在答疑
